

Computer architecture and Organization

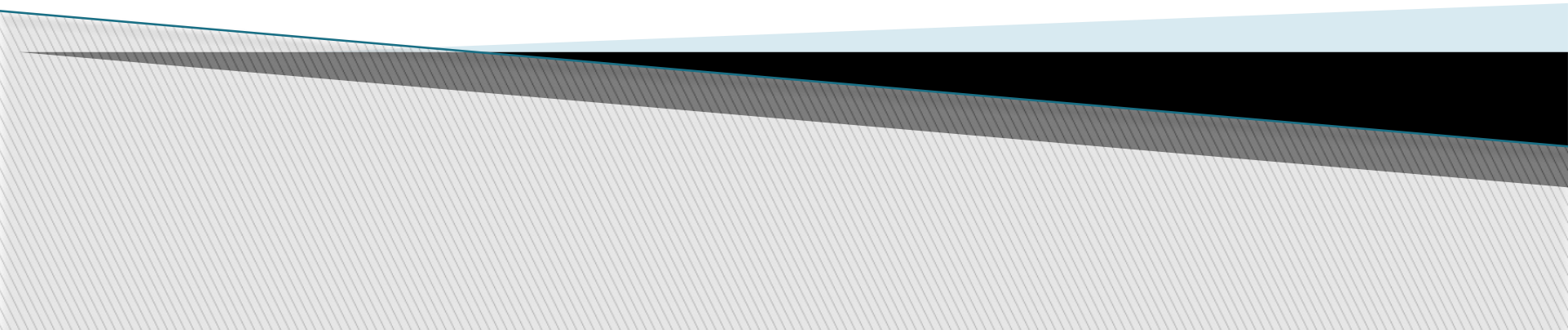
I/O Organization

M. Khademul Islam, PhD

Professor, Dept. of CSE, Rajshahi University, Bangladesh

Email: khademul.cse@ru.ac.bd, tel: +88-01727-786600

web: www.ru.ac.bd/cse/~mki

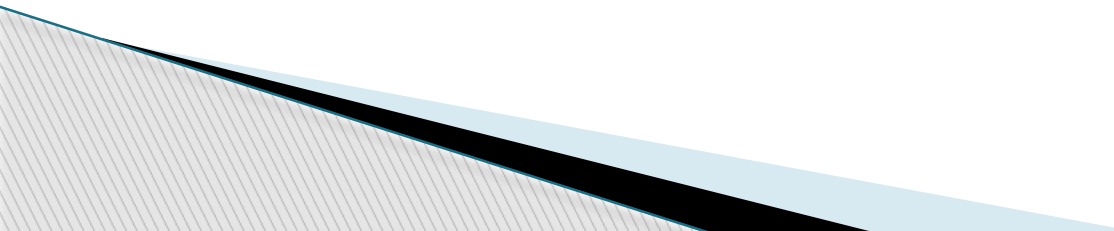
The bottom of the slide features a decorative graphic consisting of several overlapping, wavy, horizontal bands. The colors transition from a light blue at the top, through a dark grey, to a light grey at the bottom. The waves are more pronounced on the left side and fade towards the right.

I/O Organization

I/O Interface

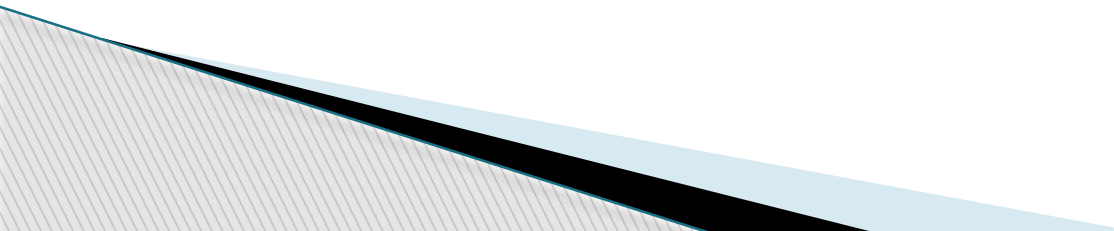
- ▶ Provides the method for transferring information between internal computing unit and external I/O devices.

Interface

- ▶ I/O interface requires the following things:
 - Special hardware components between the CPU and peripherals
 - Supervise and synchronize all input and output transfers
- 

I/O Organization

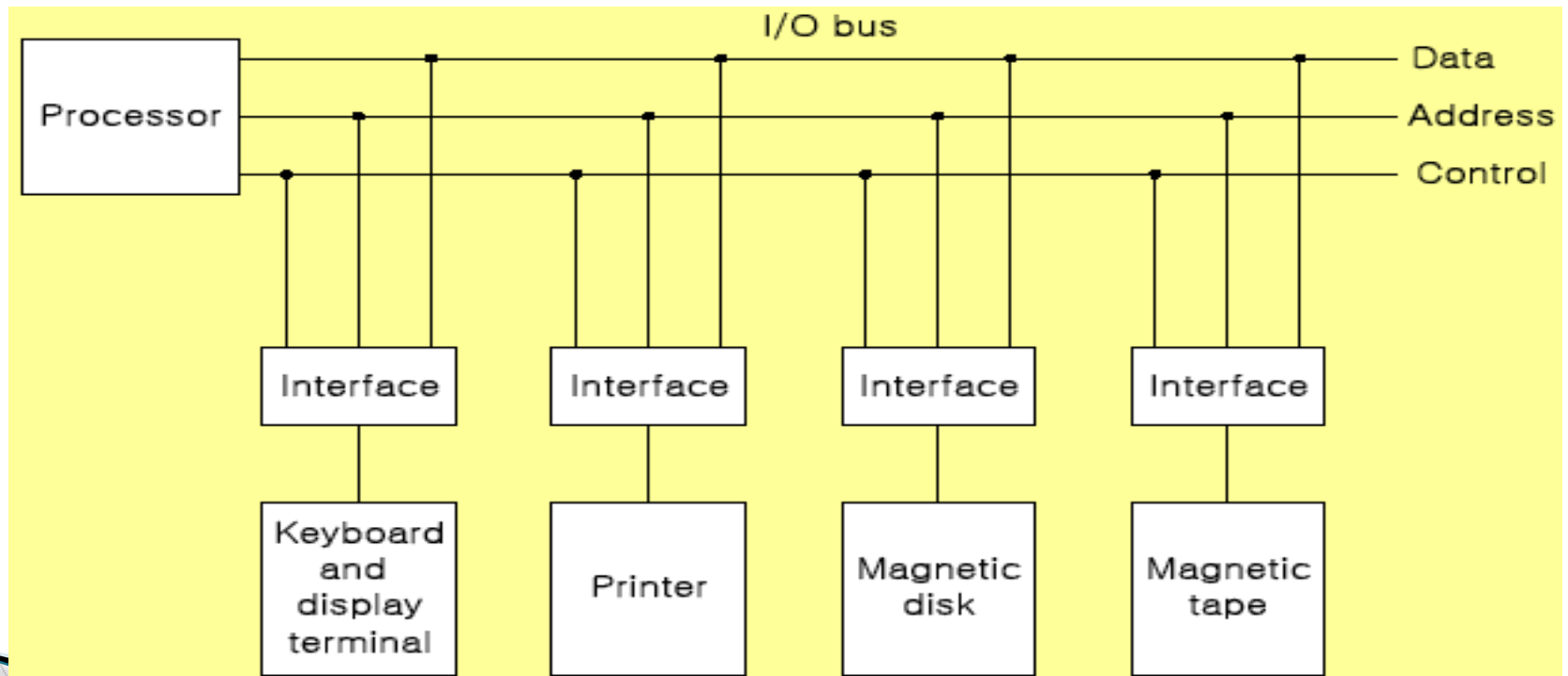
Peripheral devices

- ▶ The input or output devices attached to the computer are also called peripherals.
 - ▶ Among the most common peripherals are keyboards, display units and printers
 - ▶ The peripherals that provide auxiliary storage for the system are magnetic disks and tapes.
 - ▶ The devices that are under the direct control of computer are said to be connected online.
- 

I/O Organization

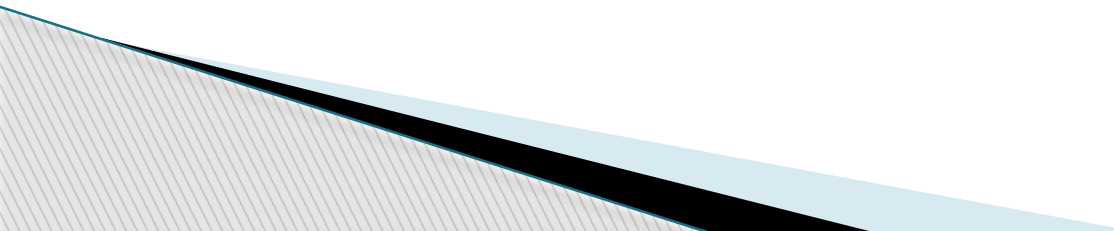
I/O Bus and Interface Modules

- ▶ The interface of I/O with CPU is performed by different I/O bus lines → Data lines, Address lines, Control lines:



I/O Organization

Interface Modules:

- ▶ SCSI (Small Computer System Interface)
 - ▶ IDE (Integrated Device Electronics)
 - ▶ RS-232
 - ▶ USB (Universal Serial Bus)
- 

I/O Organization

I/O versus Memory Bus

- ▶ Computer buses are used to communicate with memory and I/O.

Three ways to make such communication:

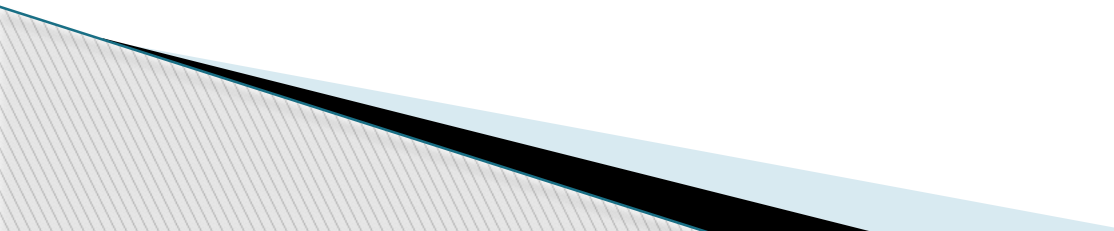
- ▶ Use two separate buses, one for memory and the other for I/O
 - ▶ Use one common bus for both memory and I/O but have separate control lines for each
 - ▶ Use one common bus for memory and I/O with common control lines
- 

I/O Organization

Synchronous Data Transfer

- ▶ All data transfers occur simultaneously during the occurrence of a clock pulse
- ▶ Registers in the interface share a common clock with CPU registers

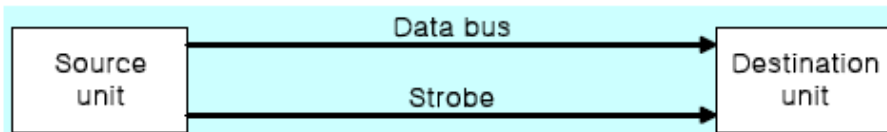
Asynchronous Data Transfer

- ▶ Internal timing in each unit (*CPU and Interface*) is independent
 - ▶ Each unit uses its own private clock for internal registers
- 

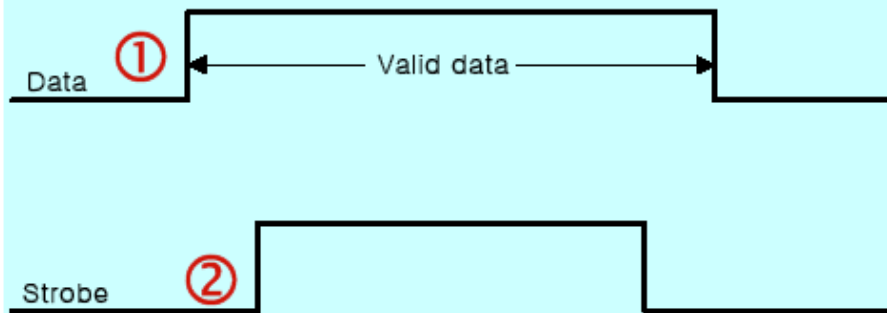
I/O Organization

Strobe : Control signal to indicate the time at which data is being transmitted

- Source-initiated strobe
- Destination-initiated strobe

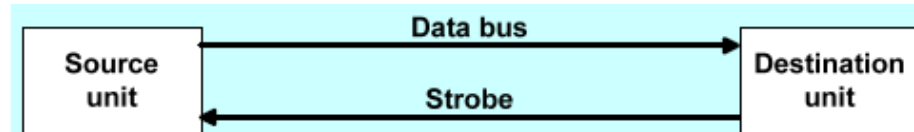


(a) Block diagram

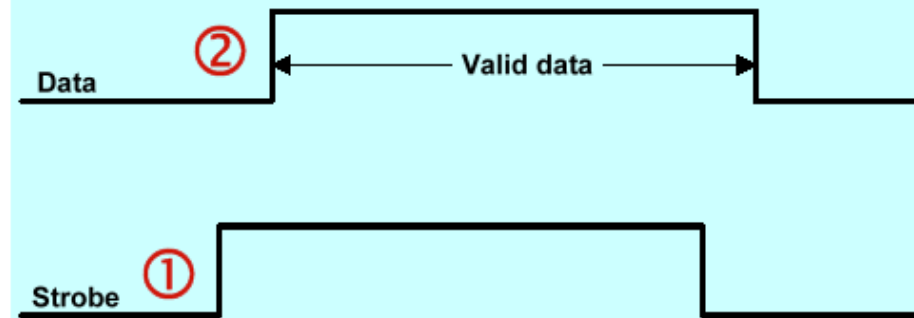


(b) Timing diagram

Source-initiated strobe



(a) Block diagram



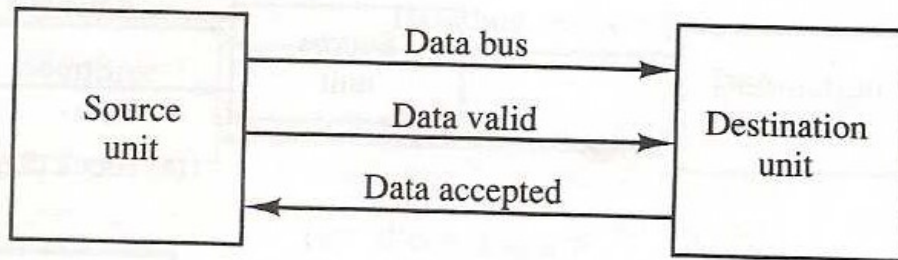
(b) Timing diagram

Destination-initiated strobe

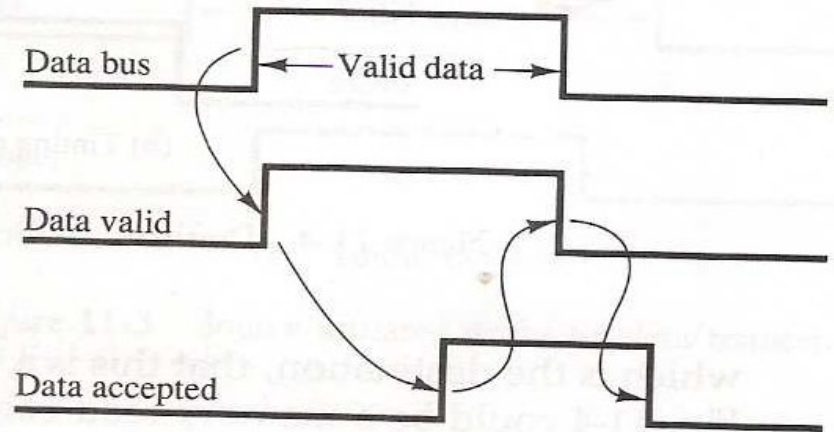
I/O Organization

Handshake: Agreement between two units

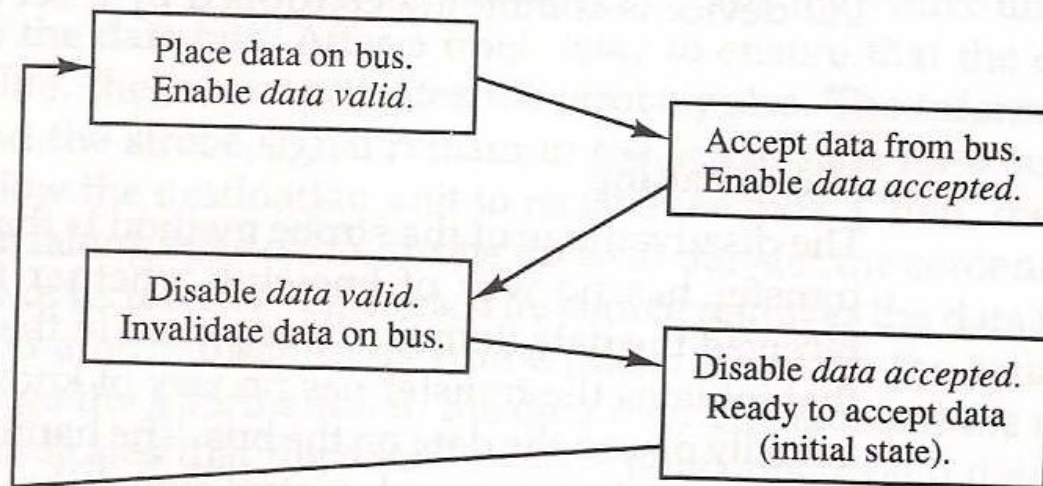
- Source-initiated



(a) Block diagram



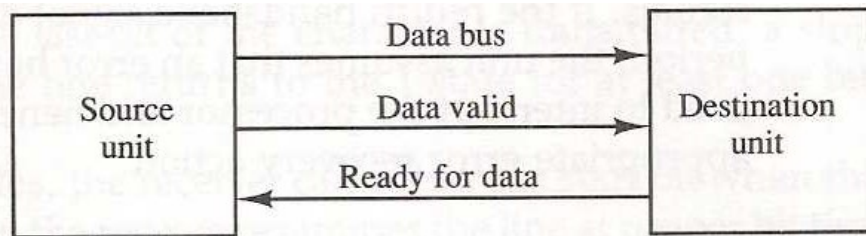
(b) Timing diagram



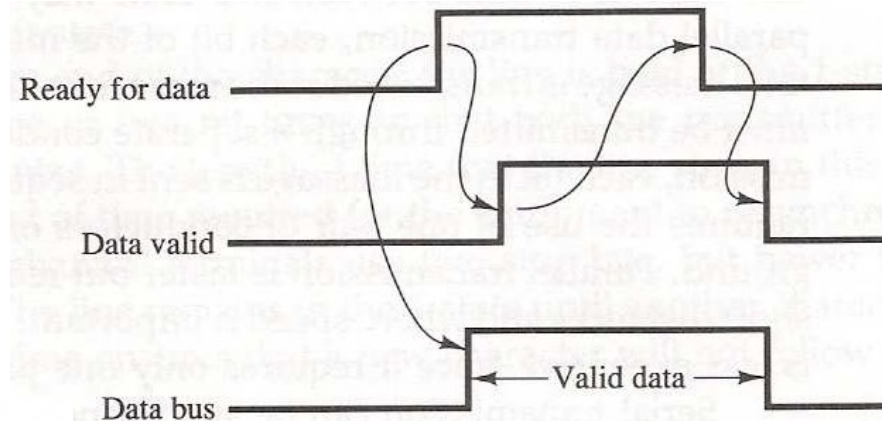
(c) Sequence of events

I/O Organization

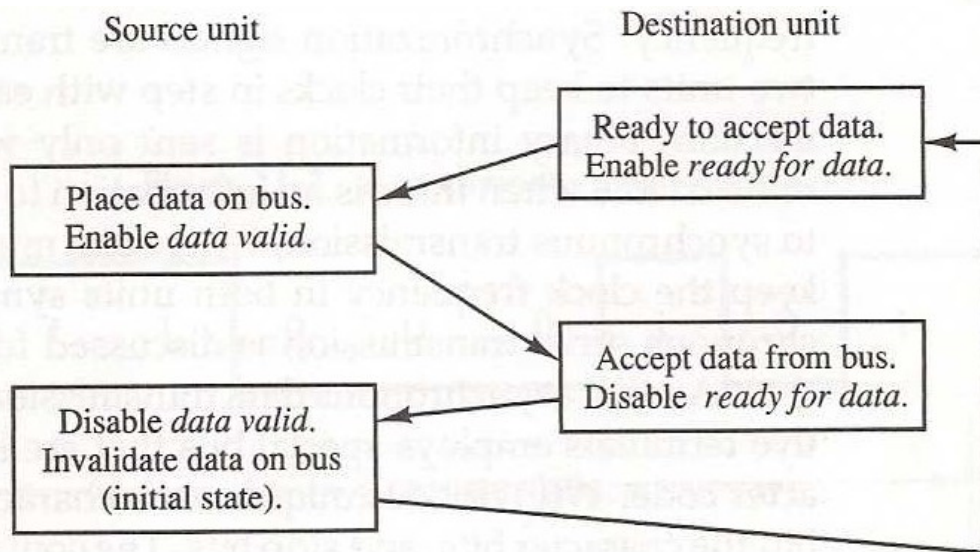
Destination initiated handshake



(a) Block diagram



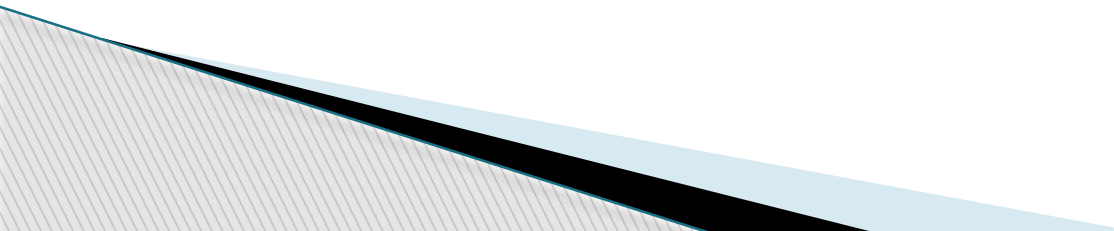
(b) Timing diagram



(c) Sequence of events

I/O Organization

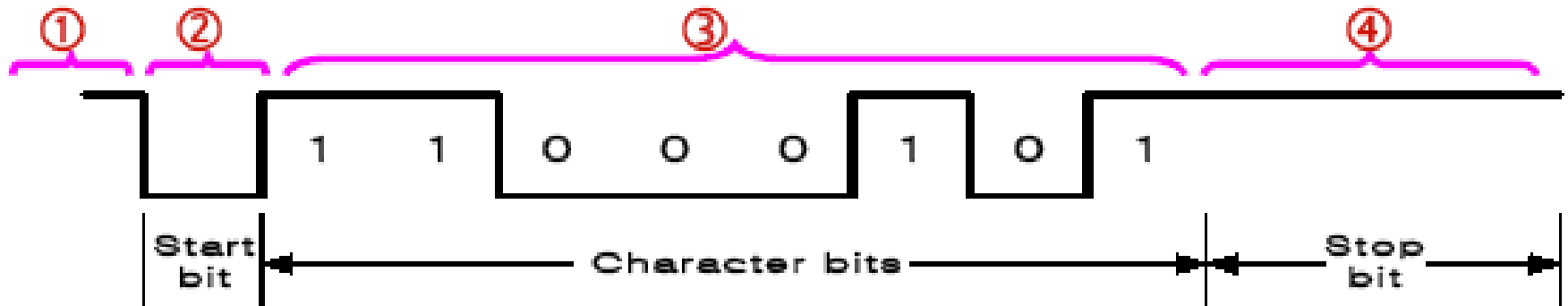
Asynchronous Data Transfer

- ▶ Special bits are inserted at both ends of the character code
 - ▶ Each character consists of three parts :
 - ▶ start bit : always "0", indicate the beginning of a character
 - ▶ character bits : data
 - ▶ stop bit : always "1"
- 

I/O Organization

Asynchronous transmission rules

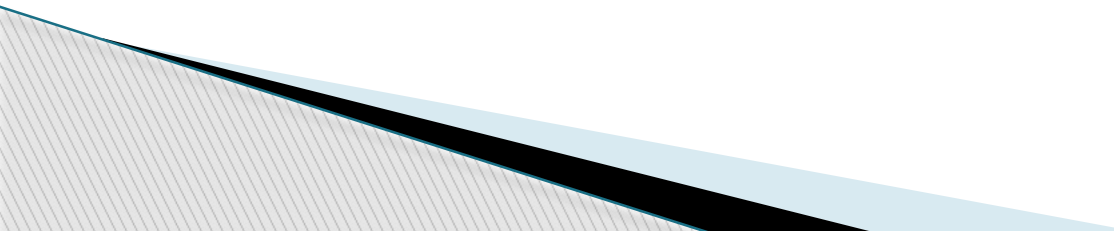
- (1) When a character is not being sent, the line is kept in the 1-state
- (2) The initiation is detected from the start bit, which is always "0"
- (3) The character always follow the start bit
- (4) A stop bit is detected when the line returns to the 1-state for at least one bit time
- UART (Universal Asynchronous Receiver): 8250



I/O Organization

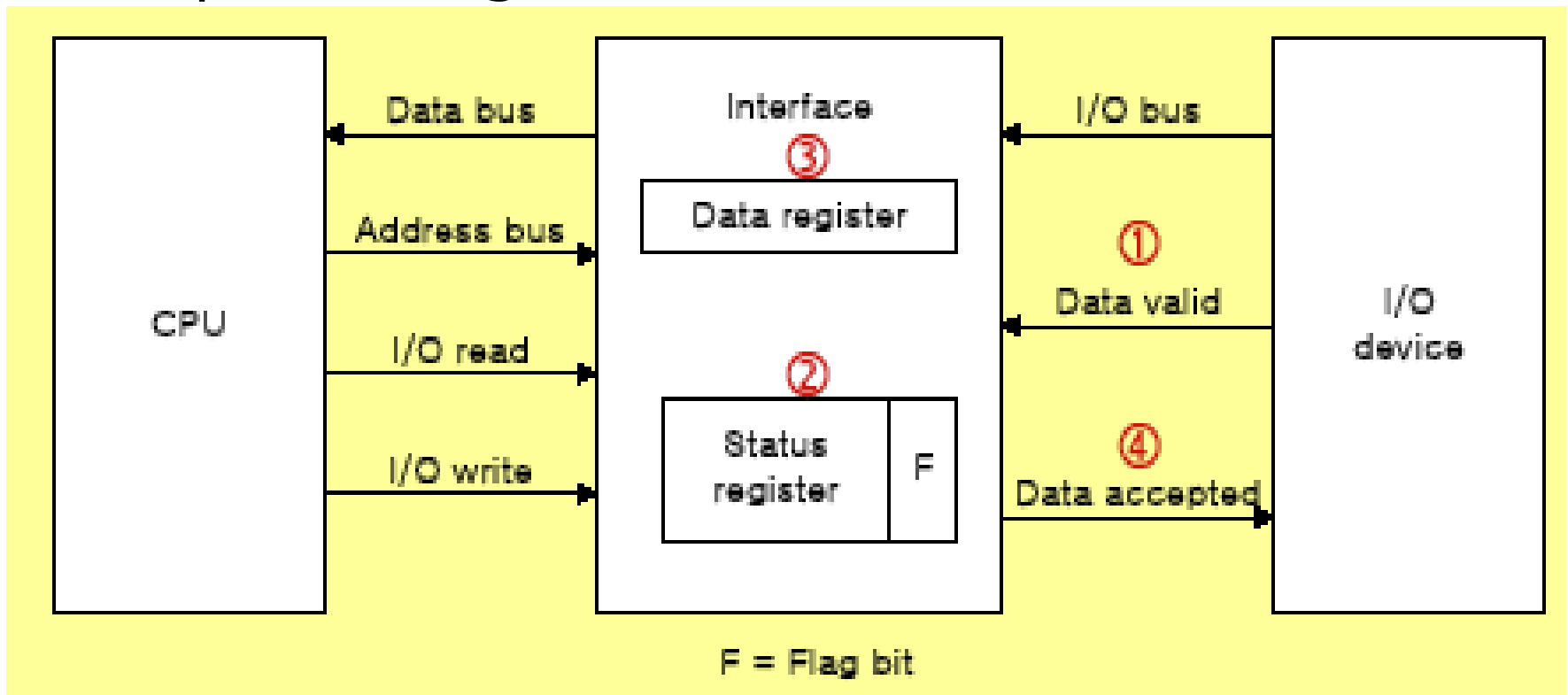
Modes of Transfer

Data transfer to and from peripherals

- ▶ 1) Programmed I/O
 - ▶ 2) Interrupt-initiated I/O
 - ▶ 3) Direct Memory Access (**DMA**)
 - ▶ 4) I/O Processor (**IOP**)
- 

I/O Organization

Example of Programmed I/O

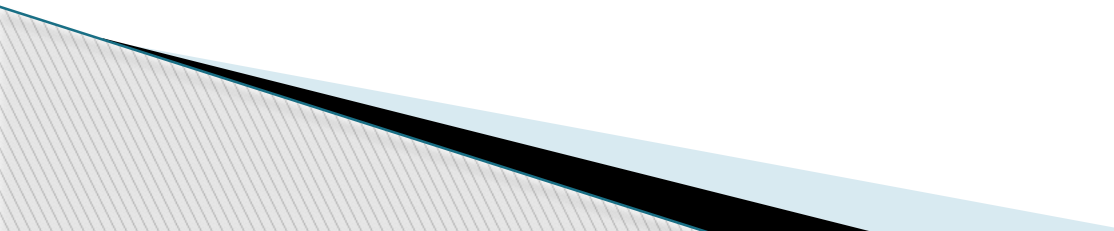


I/O Organization

Interrupt-initiated I/O

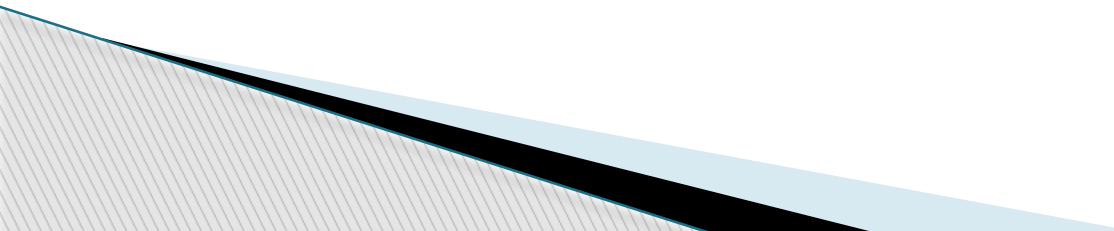
- ▶ 1) Non-vectored : fixed branch address
- ▶ 2) Vectored : interrupt source supplies the branch address

Priority Interrupt

- ▶ Identify the source of the interrupts from simultaneous several sources
 - ▶ Determine which one is to be serviced first from simultaneous multiple requests
- 

I/O Organization

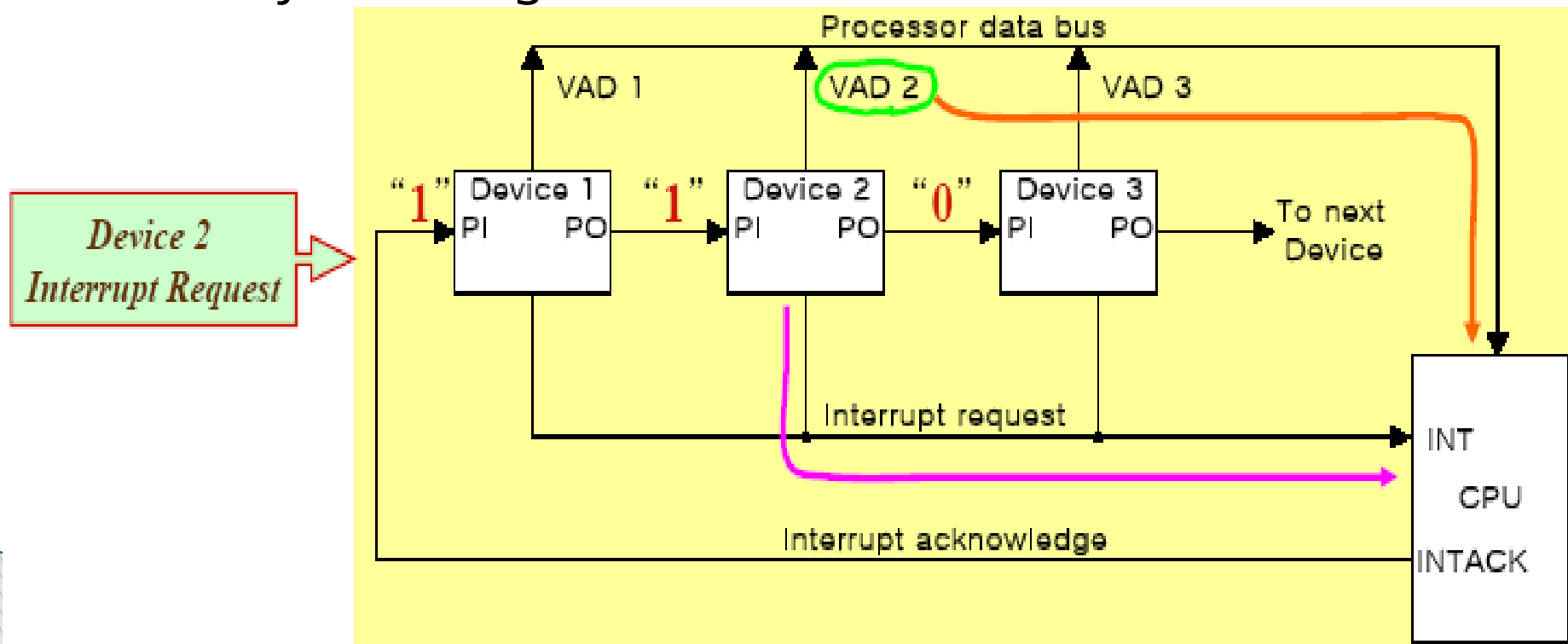
Software Polling

- ▶ Identify the highest-priority source by software means
 - ▶ Polls the interrupt sources in sequence
 - ▶ The highest-priority source is tested first
- 

I/O Organization

Hardware priority interrupt

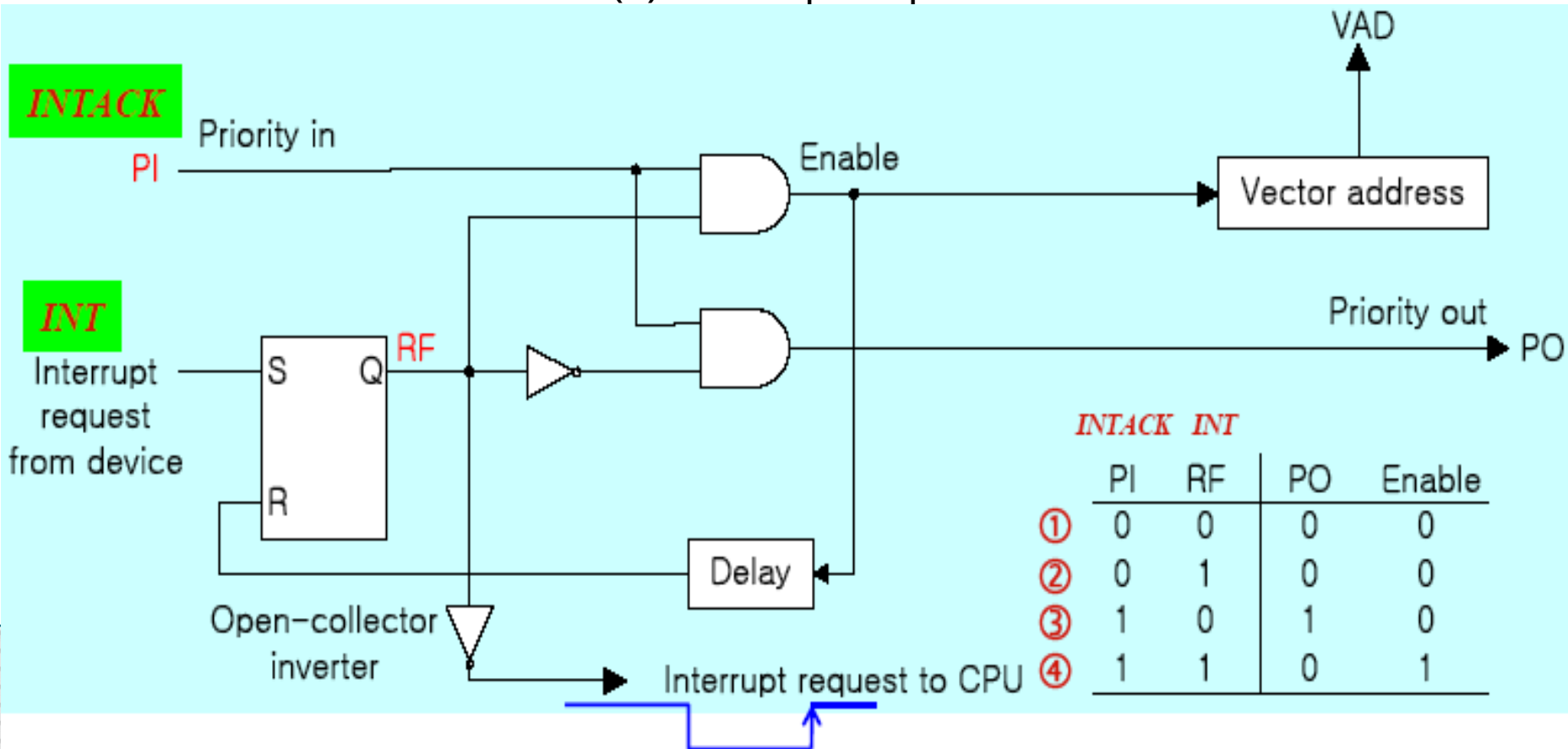
- ▶ Daisy-Chaining, parallel priority
 - Daisy -chaining



I/O Organization

► One step of DC

- (1) No interrupt request
- (2) Invalid : interrupt request, but no ack
- (3) No interrupt request: Pass to other device
- (4) Interrupt request



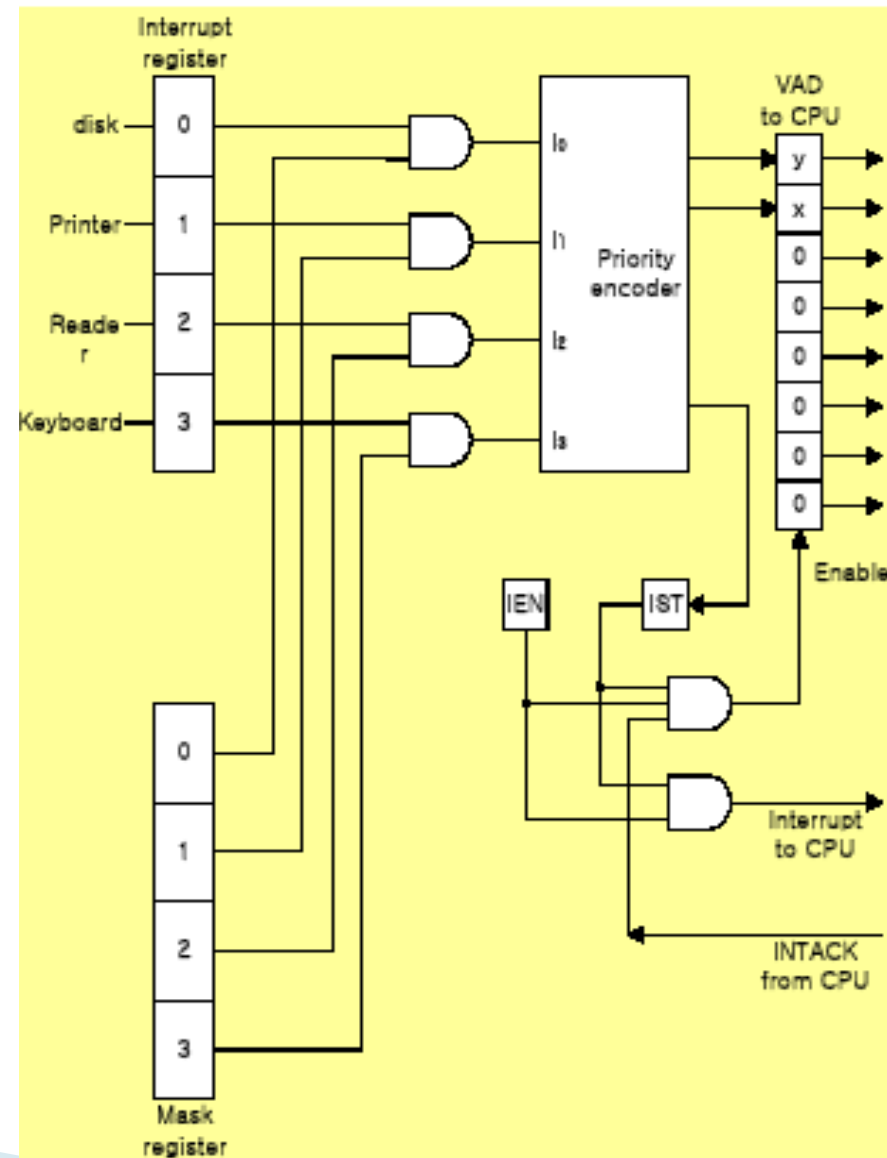
I/O Organization

Parallel Priority

► Priority Encoder

Parallel Priority :

- Interrupt Enable F/F (**IEN**): set or cleared by program
- Interrupt Status F/F (**IST**): set or cleared by output



I/O Organization

Interrupt Cycle

- ▶ At the end of each instruction cycle, CPU checks IEN (Int. Enable) and IST (Int. Status)
- ▶ if both IEN and IST equal to "1"
- ▶ CPU goes to an Interrupt Cycle
 - Sequence of micro-operation during Interrupt cycle

Branch to ISR

$SP \leftarrow SP - 1$

: Decrement stack point

$M[SP] \leftarrow PC$

: Push PC into stack

$INTACK \leftarrow 1$

: Enable INTACK

$PC \leftarrow VAD$

: Transfer VAD to PC

$IEN \leftarrow 0$

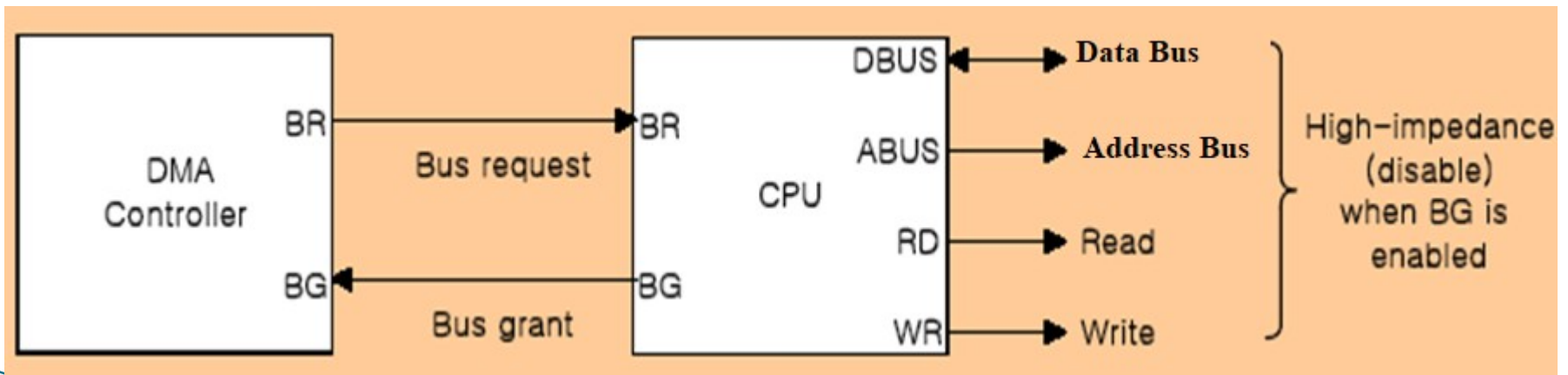
: Disable further interrupts

Go to Fetch next instruction

I/O Organization

Direct Memory Access (DMA)

- ▶ DMA controller takes over the buses to manage the transfer directly between the I/O device and memory
- ▶ Transfer Modes
 - Burst transfer : Block
 - Cycle stealing transfer : Word

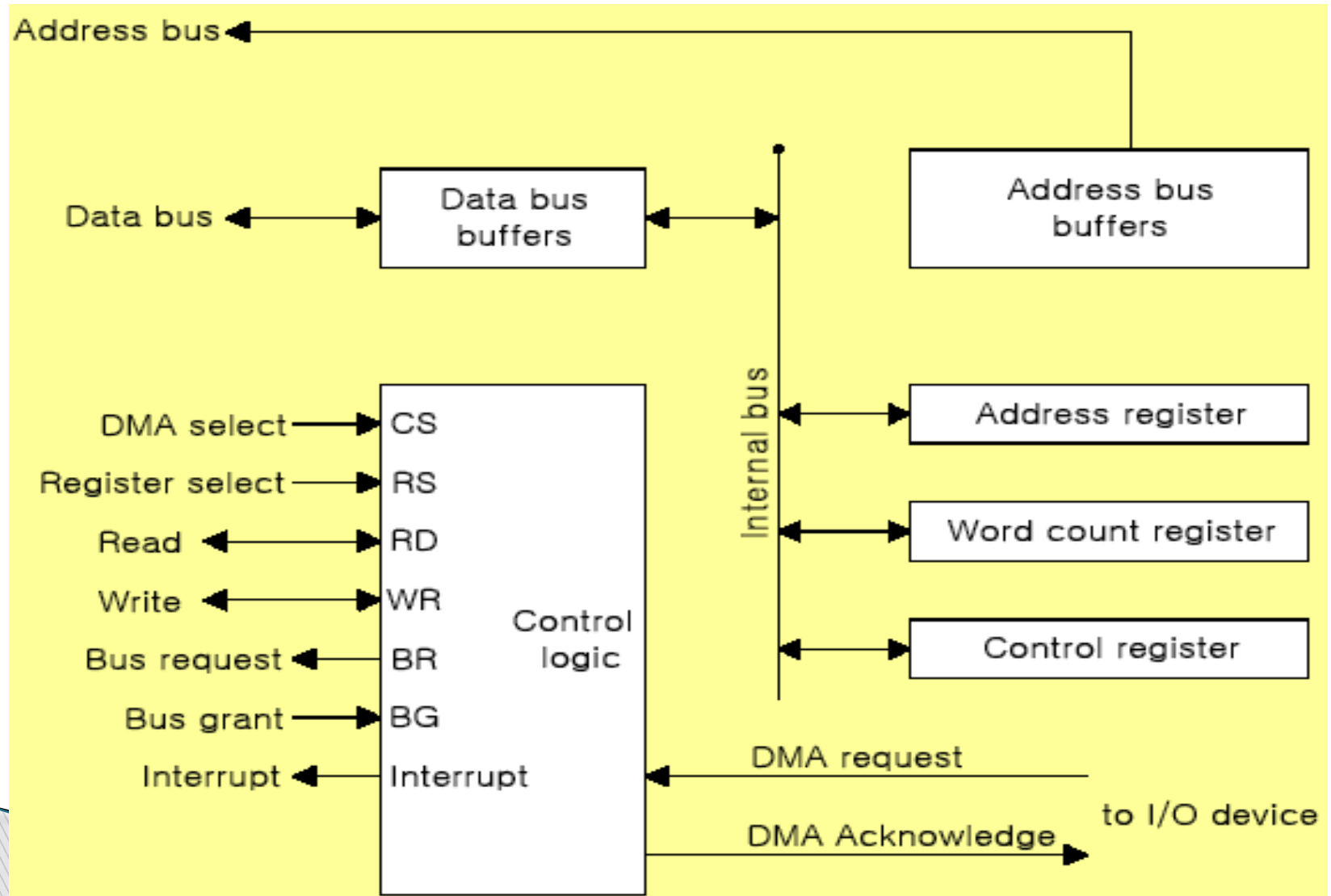


I/O Organization

DMA Controller (Intel 8237 DMAC) Initialization Process

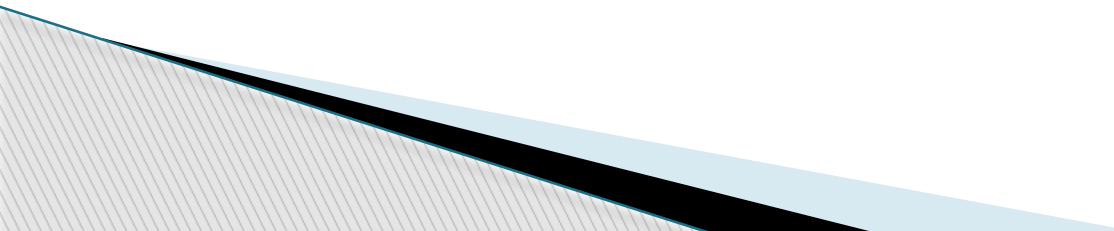
- ▶ 1) Set Address register: memory address for read/write
- ▶ 2) Set Word count register: the number of words to transfer
- ▶ 3) Set transfer mode :
 - read/write,
 - burst/cycle stealing,
 - I/O to I/O,
 - I/O to Memory,
 - Memory to Memory
 - Memory search
 - I/O search
- ▶ 4) DMA transfer start : *next section*
- ▶ 5) EOT (End of Transfer): Interrupt

I/O Organization

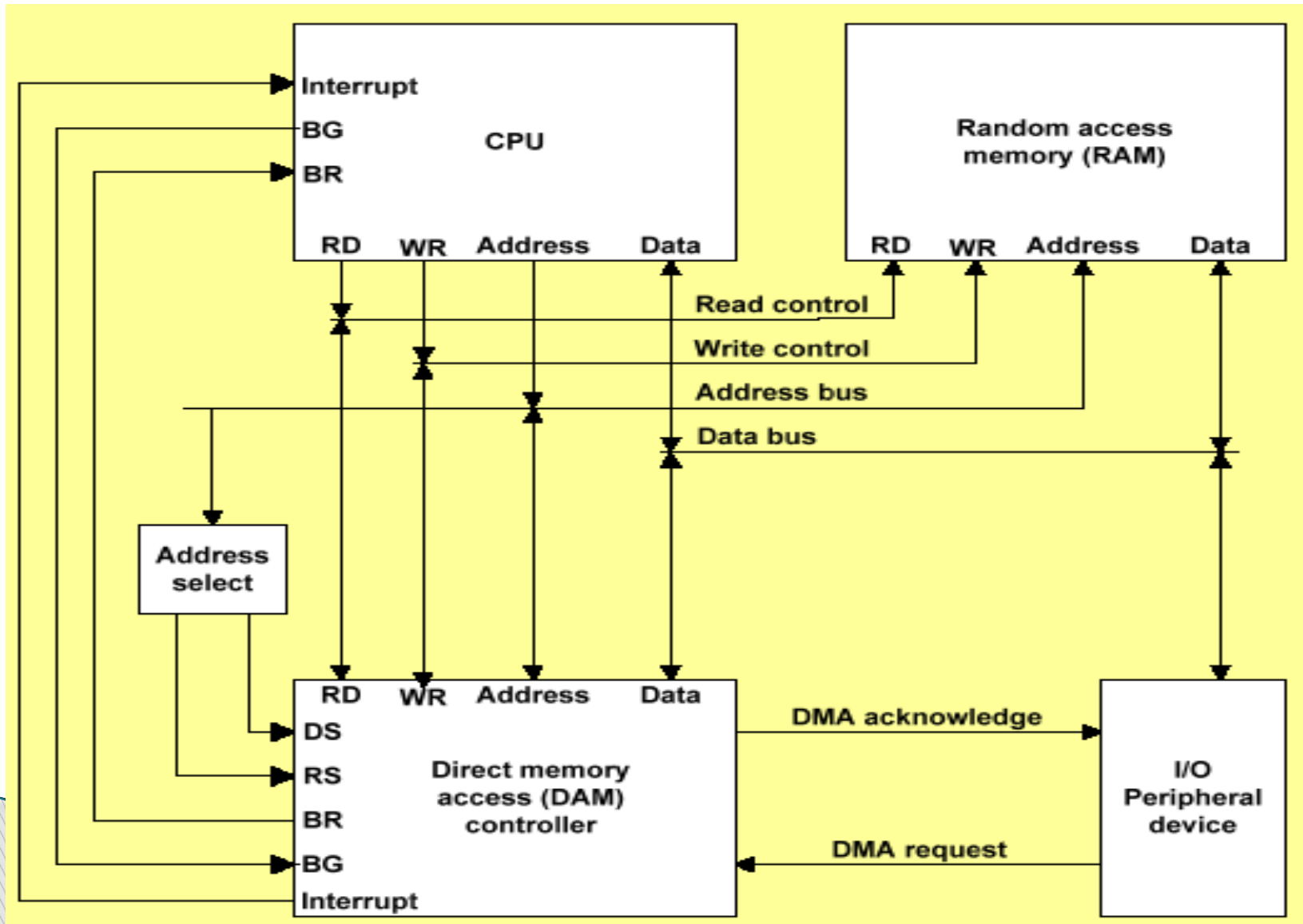


I/O Organization

DMA Transfer (I/O to Memory)

- ▶ 1) I/O Device sends a DMA request
 - ▶ 2) DMAC activates the **BR line**
 - ▶ 3) CPU responds with **BG line**
 - ▶ 4) DMAC sends a DMA acknowledge to the I/O device
 - ▶ 5) I/O device puts a word in data bus (*memory write*)
 - ▶ 6) DMAC write a data to the address specified by **AR**
 - ▶ 7) Decrement Word count register
 - ▶ 8) Word count register = 0 (EOT interrupt)
 - ▶ 9) Word count register $\neq$ 0 (DMAC checks the DMA request from I/O device)
- 

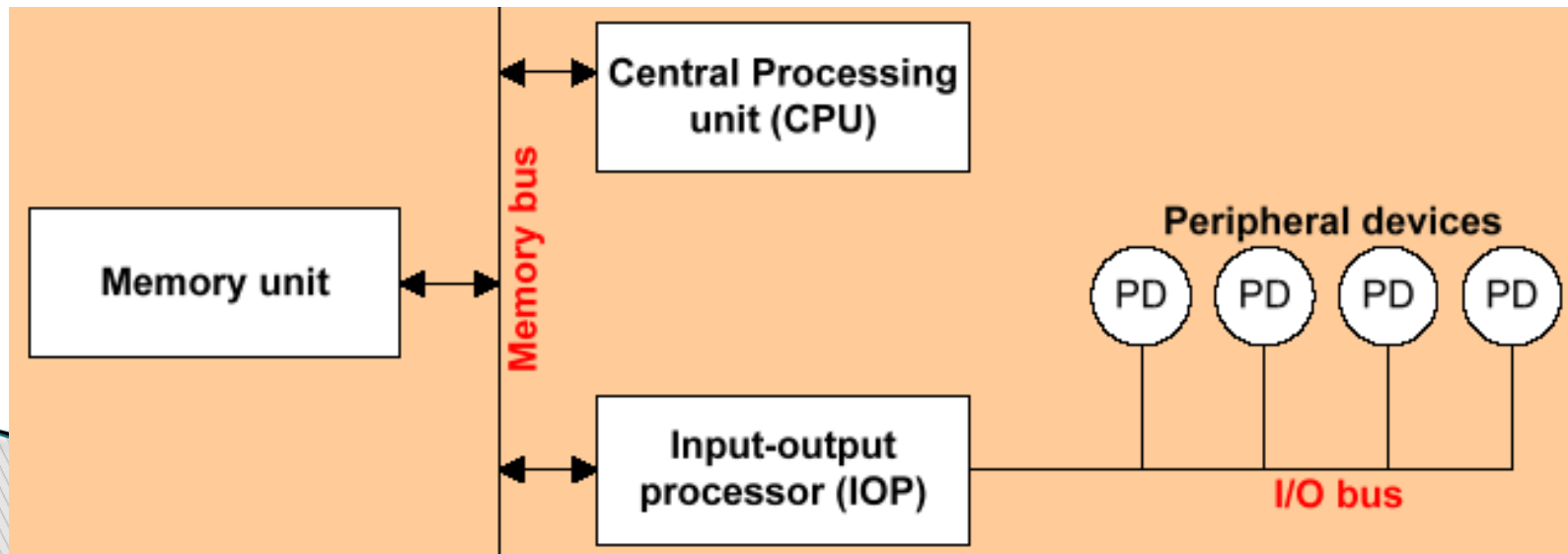
I/O Organization



I/O Organization

Input-Output Processor (IOP)

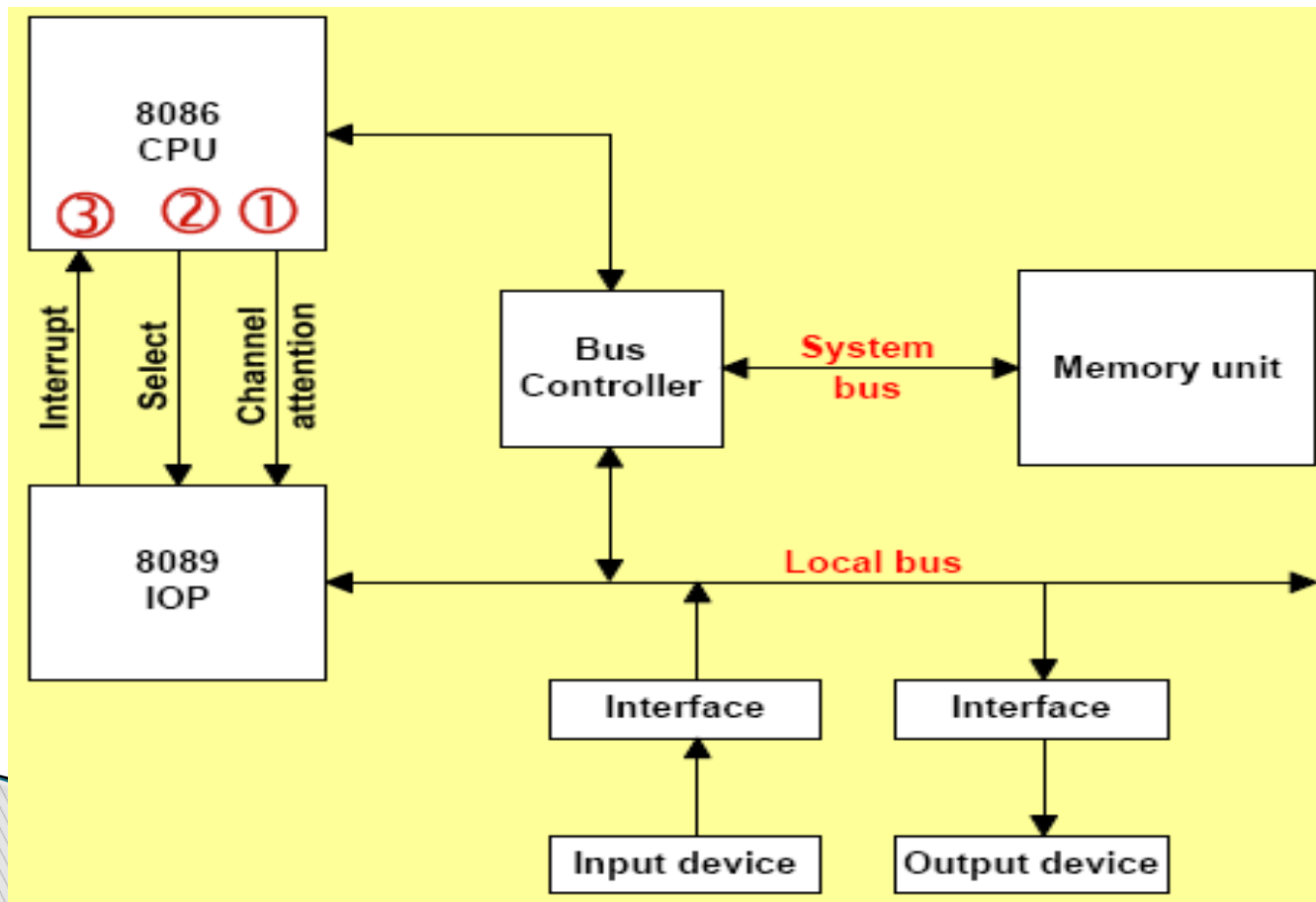
- ▶ Communicate directly with all I/O devices
- ▶ Fetch and execute its own instruction
 - IOP instructions are designed for I/O transfer
 - DMAC must be set up entirely by the CPU
- ▶ Designed to handle the details of I/O processing



I/O Organization

Intel 8089 IOP

- ① CPU enables channel attention
- ② Select one of two channels of 8089
- ③ 8089 gets attention of the CPU by sending an interrupt request



Bus Arbitration

Bus arbitration is required to decide which device gets control of the bus when multiple devices request it at the same time.

Why need?

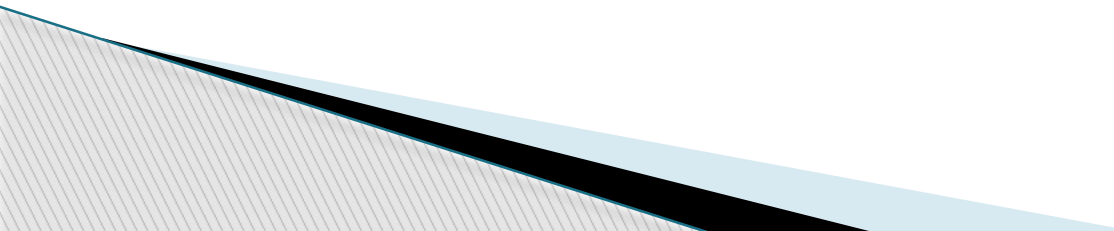
❑ To avoid conflicts

- If two devices drive the bus at the same time, data will be corrupted.
- Arbitration ensures that only one master uses the bus at a time.

❑ To maintain system stability

- If multiple masters try to access the bus simultaneously, the system may hang.

❑ To ensure fair access

- All devices should get a fair chance to use the bus.
 - The device that requests first or has higher priority gets access.
- 

Bus Arbitration

Why need?

❑ To control priority

- Some devices (e.g., DMA controller) need higher priority.

❑ To handle parallel requests

- When many devices request the bus at the same time, the arbitration algorithm decides who will get access.

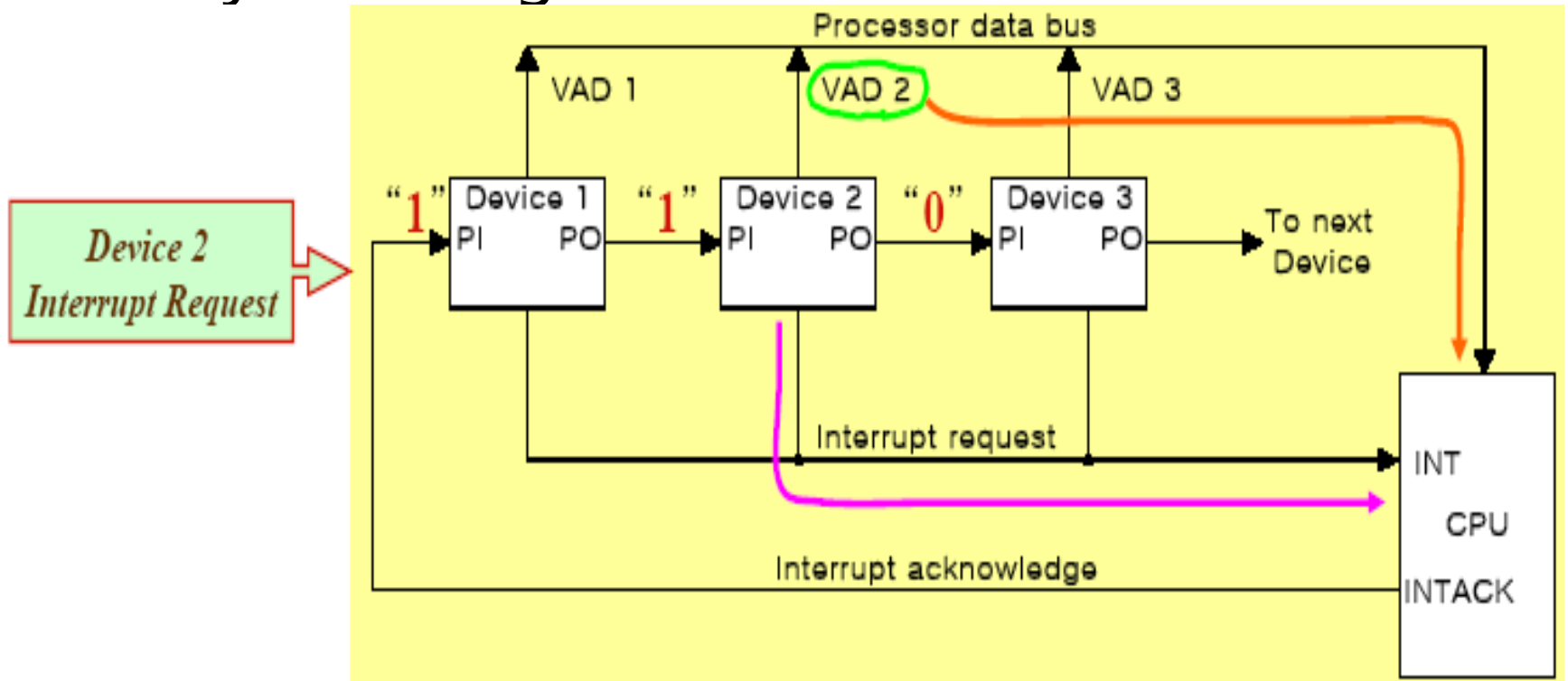
Bus Arbitration

Bus Arbitration Techniques

- ☐ Daisy Chaining
- ☐ Polling
- ☐ Independent Request

Bus Arbitration

□ Daisy Chaining



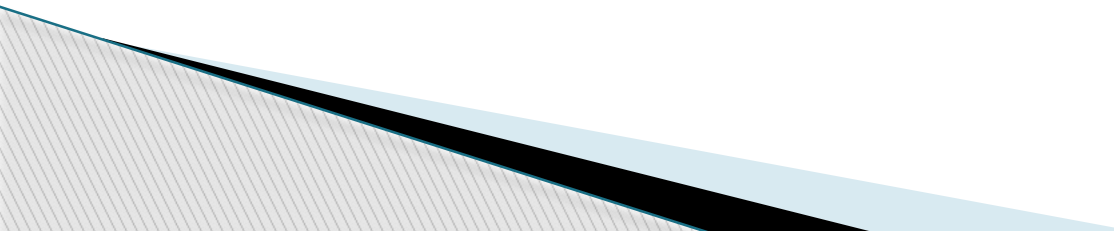
Bus Arbitration

❑ Daisy Chaining

❖ Advantages

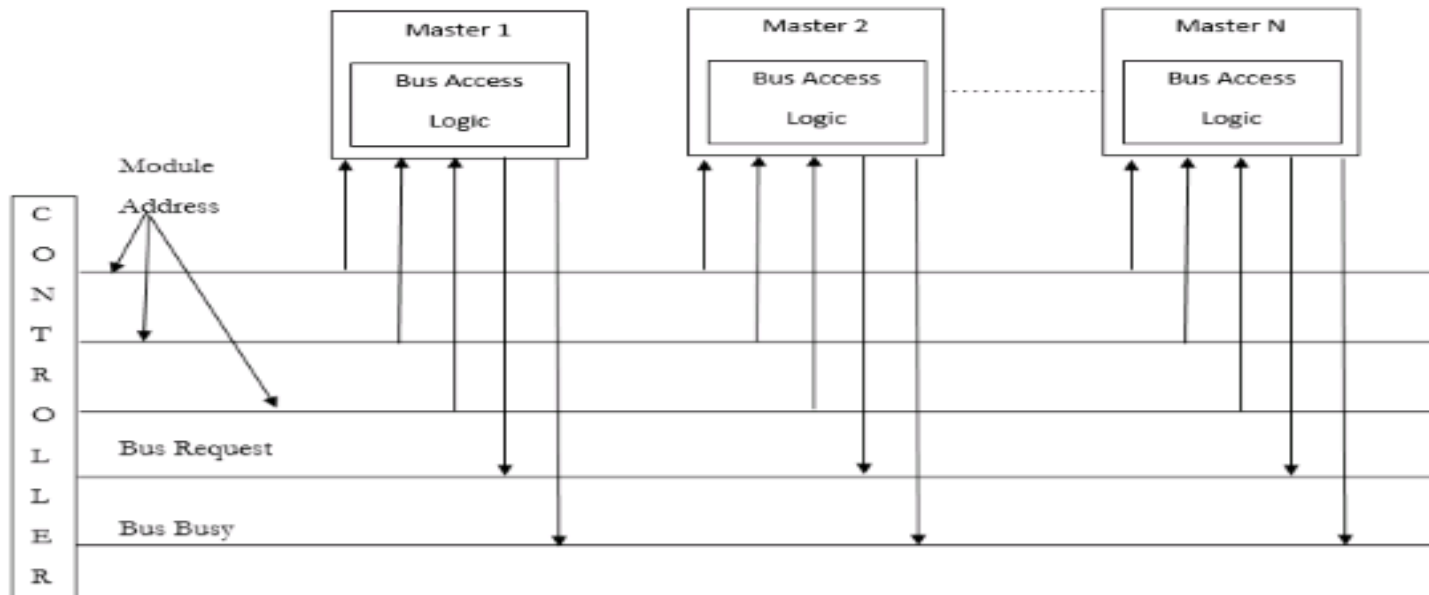
- Simple design
- The user can add more devices anywhere along the chain

❖ Disadvantages

- Priority is fixed
 - Lower-priority devices may face starvation
 - Delay increases in long chains
 - If one device fails then the entire system will stop working.
- 

Bus Arbitration

□ Polling



In this, the controller is used to generate the address for the master(unique priority), the number of address lines required depends on the number of masters

Bus Arbitration

□ Polling

❖ Advantages

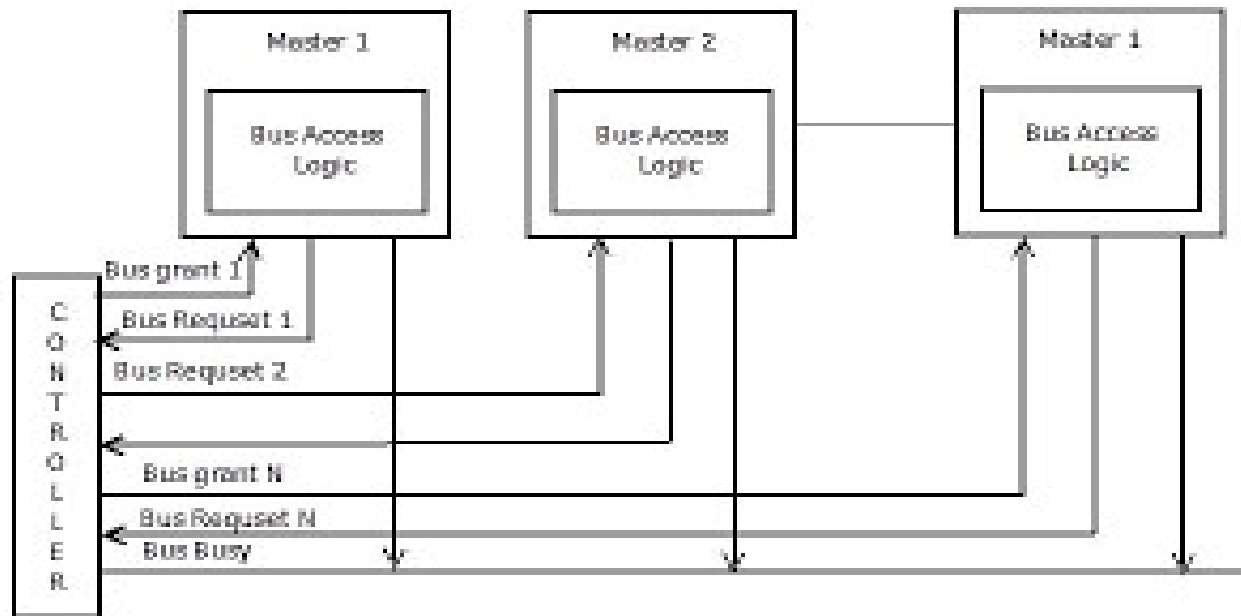
- This method does not favor any particular device and processor.
- The method is also quite simple.
- If one device fails then the entire system will not stop working.
- To reduce the starvation problem.

❖ Disadvantages

- Adding bus masters is difficult as increases the number of address lines of the circuit.

Bus Arbitration

□ Independent Request



The built-in priority decoder within the controller selects the highest priority request and asserts the corresponding bus grant signal.

Bus Arbitration

□ Independent Request

❖ Advantages

- Priority control is easy
- This method generates a fast response.
- To reduce the starvation problem.

❖ Disadvantages

- Hardware cost is high as a large no. of control lines is required.